

LAPORAN PRAKTIKUM
STRUKTUR DATA
“laporan Praktikum Pekan 6”

Disusun Oleh:

Faiz Fikri Satria

2511533026

Dosen Pengampu : Dr. Wahyudi, S.T, M.T.
Asisten Praktikum : Sherly Sukmadira



DAPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan praktikum dan laporan ini dengan baik. Laporan praktikum ini disusun sebagai dokumentasi dari praktikum pekan ke-6 mata kuliah Struktur Data mengenai implementasi **Doubly Linked List (DLL)** menggunakan bahasa pemrograman Java. Praktikum ini membahas pembuatan node dua arah, operasi penambahan data di awal, akhir, dan posisi tertentu, penelusuran maju dan mundur, serta operasi penghapusan node. Melalui praktikum ini, kami mempelajari keunggulan Doubly Linked List dibandingkan Singly Linked List, terutama kemampuan traversal dua arah dan penghapusan node yang lebih efisien.

Kami mengucapkan terima kasih yang sebesar-besarnya kepada Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu yang telah memberikan materi dan penjelasan yang mendalam, serta Sherly Sukmadira selaku asisten praktikum yang selalu siap membantu dan membimbing selama proses praktikum. Terima kasih juga kami sampaikan kepada rekan-rekan mahasiswa yang telah saling mendukung dan berdiskusi dalam menyelesaikan tugas praktikum ini. Praktikum Doubly Linked List ini sangat penting untuk memahami konsep pointer ganda dan manajemen memori yang lebih kompleks.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, segala kritik dan saran yang membangun sangat kami harapkan untuk penyempurnaan laporan di masa yang akan datang.

Padang, 13 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR	ii
DAFTAR ISI	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 NodeDLL_2511533026	2
2.1.1 Teori.....	2
2.1.2 Langkah-langkah	2
2.1.3 Contoh Kode	2
2.1.5 Analisis	2
2.2 InsertDLL_2511533026	3
2.2.1 Teori	3
2.2.2 Langkah-langkah	3
2.2.3 Contoh Kode	3
2.2.4 Hasil Output	4
2.2.5 Analisis	4
2.3 PenelusuranDLL_2511533026	4
2.2.1 Teori	4
2.2.2 Langkah-langkah	4

2.2.3 Contoh Kode	4
2.2.4 Hasil Output	5
2.2.5 Analisis	5
2.4 HapusDLL_2511533026	5
2.2.1 Teori	5
2.2.2 Langkah-langkah	5
2.2.3 Contoh Kode	6
2.2.4 Hasil	6
2.2.5 Analisis	6
2.6 Tugas Pekan 6	7
2.5.1. Analisis	7
BAB III KESIMPULAN	9
DAFTAR PUSTAKA	110

BAB I

PENDAHULUAN

1.1 Latar Belakang

Doubly Linked List (DLL) adalah struktur data linier di mana setiap node memiliki dua pointer, yaitu pointer ke node berikutnya (next) dan pointer ke node sebelumnya (prev). Hal ini memungkinkan traversal data baik secara maju maupun mundur. Praktikum pekan ini melanjutkan materi Singly Linked List dengan menambahkan kemampuan dua arah pada linked list.

Praktikum ini mencakup implementasi manual node DLL, operasi insert, traversal dua arah, dan operasi delete pada berbagai posisi.

1.2 Tujuan

1. Memahami konsep dan struktur Doubly Linked List beserta perbedaannya dengan Singly Linked List.
2. Mengimplementasikan class Node dengan dua pointer (next dan prev).
3. Mengimplementasikan operasi penambahan node di awal, akhir, dan posisi tertentu.
4. Mengimplementasikan penelusuran data secara maju (forward) dan mundur (backward).
5. Mengimplementasikan operasi penghapusan node di awal, akhir, dan posisi tertentu.

1.3 Manfaat

1. Mahasiswa memahami kelebihan Doubly Linked List dalam hal fleksibilitas traversal dan penghapusan data.
2. Meningkatkan kemampuan dalam mengelola pointer ganda dan menghindari kesalahan referensi.
3. Memberikan fondasi yang kuat untuk memahami struktur data yang lebih kompleks seperti Deque dan Tree.

BAB II

PEMBAHASAN

2.1 NodeDLL_2511533026

2.1.1 Teori

Node pada Doubly Linked List memiliki tiga komponen: data, pointer ke node berikutnya (next), dan pointer ke node sebelumnya (prev).

2.1.2 Langkah-langkah

1. Membuat class NodeDLL_2511533026.
2. Mendeklarasikan variabel data_3026, next_3026, dan prev_3026.
3. Membuat konstruktor untuk inisialisasi data dan kedua pointer.

2.1.3 Contoh Kode

```
1 package Peko0_2511533026;
2
3 public class NodeDll_2511533026 {
4     // menginisialisasi kelas Node
5     int data_3026; // data
6     NodeDll_2511533026 next_3026; // Pointer ke next node
7     NodeDll_2511533026 prev_3026; // Pointer ke previous node
8
9     //konstruktor
10    public NodeDll_2511533026 (int data_3026) {
11        this.data_3026 = data_3026;
12        this.next_3026 = null;
13        this.prev_3026 = null;
14    }
15 }
```

2.1.4 Analisis

Penambahan pointer prev memungkinkan akses dua arah, sehingga operasi delete dan traversal menjadi lebih efisien dibandingkan SLL.

2.2 InsertDLL_2511533026

2.2.1 Teori

Operasi insert pada DLL meliputi insertBegin ($O(1)$), insertEnd, dan insertAtPosition. Setiap insert harus memperbarui kedua pointer (next dan prev).

2.2.2 Langkah-langkah

1. Implementasi insertBegin.
2. Implementasi insertEnd dengan traversal.
3. Implementasi insertAtPosition dengan pengecekan posisi.

2.2.3 Contoh Kode

```

1 // Metodo: Inserta nuovo nodo alla fine
2
3 #include <iostream>
4 #include <string>
5 #include <vector>
6 #include <list>
7 #include <map>
8 #include <algorithm>
9 #include <stack>
10 #include <queue>
11 #include <deque>
12 #include <set>
13 #include <unordered_set>
14 #include <map>
15 #include <unordered_map>
16 #include <vector>
17 #include <string>
18 #include <sstream>
19 #include <fstream>
20 #include <memory>
21 #include <limits>
22 #include <climits>
23 #include <chrono>
24 #include <ctime>
25 #include <cmath>
26 #include <cstdlib>
27 #include <cstdio>
28 #include <cstring>
29 #include <csignal>
30 #include <unistd.h>
31 #include <sys/types.h>
32 #include <sys/stat.h>
33 #include <fcntl.h>
34 #include <sys/time.h>
35 #include <sys/resource.h>
36 #include <sys/mman.h>
37 #include <sys/wait.h>
38 #include <sys/socket.h>
39 #include <sys/un.h>
40 #include <sys/poll.h>
41 #include <sys/epoll.h>
42 #include <sys/ioctl.h>
43 #include <sys/uio.h>
44 #include <sys/sysmacros.h>
45 #include <sys/statvfs.h>
46 #include <sys/fsuid.h>
47 #include <sys/xattr.h>
48 #include <sys/sendfile.h>
49 #include <sys/mount.h>
50 #include <sys/quota.h>
51 #include <sys/swap.h>
52 #include <sys/sem.h>
53 #include <sys/shm.h>
54 #include <sys/ipc.h>
55 #include <sys/msg.h>
56 #include <sys/types.h>
57 #include <sys/time.h>
58 #include <sys/resource.h>
59 #include <sys/mman.h>
60 #include <sys/wait.h>
61 #include <sys/socket.h>
62 #include <sys/un.h>
63 #include <sys/poll.h>
64 #include <sys/epoll.h>
65 #include <sys/ioctl.h>
66 #include <sys/uio.h>
67 #include <sys/sysmacros.h>
68 #include <sys/statvfs.h>
69 #include <sys/fsuid.h>
70 #include <sys/xattr.h>
71 #include <sys/sendfile.h>
72 #include <sys/mount.h>
73 #include <sys/quota.h>
74 #include <sys/swap.h>
75 #include <sys/sem.h>
76 #include <sys/shm.h>
77 #include <sys/ipc.h>
78 #include <sys/msg.h>
79 #include <sys/types.h>
80 #include <sys/time.h>
81 #include <sys/resource.h>
82 #include <sys/mman.h>
83 #include <sys/wait.h>
84 #include <sys/socket.h>
85 #include <sys/un.h>
86 #include <sys/poll.h>
87 #include <sys/epoll.h>
88 #include <sys/ioctl.h>
89 #include <sys/uio.h>
90 #include <sys/sysmacros.h>
91 #include <sys/statvfs.h>
92 #include <sys/fsuid.h>
93 #include <sys/xattr.h>
94 #include <sys/sendfile.h>
95 #include <sys/mount.h>
96 #include <sys/quota.h>
97 #include <sys/swap.h>
98 #include <sys/sem.h>
99 #include <sys/shm.h>
100 #include <sys/ipc.h>
101 #include <sys/msg.h>
102 #include <sys/types.h>
103 #include <sys/time.h>
104 #include <sys/resource.h>
105 #include <sys/mman.h>
106 #include <sys/wait.h>
107 #include <sys/socket.h>
108 #include <sys/un.h>
109 #include <sys/poll.h>
110 #include <sys/epoll.h>
111 #include <sys/ioctl.h>
112 #include <sys/uio.h>
113 #include <sys/sysmacros.h>
114 #include <sys/statvfs.h>
115 #include <sys/fsuid.h>
116 #include <sys/xattr.h>
117 #include <sys/sendfile.h>
118 #include <sys/mount.h>
119 #include <sys/quota.h>
120 #include <sys/swap.h>
121 #include <sys/sem.h>
122 #include <sys/shm.h>
123 #include <sys/ipc.h>
124 #include <sys/msg.h>
125 #include <sys/types.h>
126 #include <sys/time.h>
127 #include <sys/resource.h>
128 #include <sys/mman.h>
129 #include <sys/wait.h>
130 #include <sys/socket.h>
131 #include <sys/un.h>
132 #include <sys/poll.h>
133 #include <sys/epoll.h>
134 #include <sys/ioctl.h>
135 #include <sys/uio.h>
136 #include <sys/sysmacros.h>
137 #include <sys/statvfs.h>
138 #include <sys/fsuid.h>
139 #include <sys/xattr.h>
140 #include <sys/sendfile.h>
141 #include <sys/mount.h>
142 #include <sys/quota.h>
143 #include <sys/swap.h>
144 #include <sys/sem.h>
145 #include <sys/shm.h>
146 #include <sys/ipc.h>
147 #include <sys/msg.h>
148 #include <sys/types.h>
149 #include <sys/time.h>
150 #include <sys/resource.h>
151 #include <sys/mman.h>
152 #include <sys/wait.h>
153 #include <sys/socket.h>
154 #include <sys/un.h>
155 #include <sys/poll.h>
156 #include <sys/epoll.h>
157 #include <sys/ioctl.h>
158 #include <sys/uio.h>
159 #include <sys/sysmacros.h>
160 #include <sys/statvfs.h>
161 #include <sys/fsuid.h>
162 #include <sys/xattr.h>
163 #include <sys/sendfile.h>
164 #include <sys/mount.h>
165 #include <sys/quota.h>
166 #include <sys/swap.h>
167 #include <sys/sem.h>
168 #include <sys/shm.h>
169 #include <sys/ipc.h>
170 #include <sys/msg.h>
171 #include <sys/types.h>
172 #include <sys/time.h>
173 #include <sys/resource.h>
174 #include <sys/mman.h>
175 #include <sys/wait.h>
176 #include <sys/socket.h>
177 #include <sys/un.h>
178 #include <sys/poll.h>
179 #include <sys/epoll.h>
180 #include <sys/ioctl.h>
181 #include <sys/uio.h>
182 #include <sys/sysmacros.h>
183 #include <sys/statvfs.h>
184 #include <sys/fsuid.h>
185 #include <sys/xattr.h>
186 #include <sys/sendfile.h>
187 #include <sys/mount.h>
188 #include <sys/quota.h>
189 #include <sys/swap.h>
190 #include <sys/sem.h>
191 #include <sys/shm.h>
192 #include <sys/ipc.h>
193 #include <sys/msg.h>
194 #include <sys/types.h>
195 #include <sys/time.h>
196 #include <sys/resource.h>
197 #include <sys/mman.h>
198 #include <sys/wait.h>
199 #include <sys/socket.h>
200 #include <sys/un.h>
201 #include <sys/poll.h>
202 #include <sys/epoll.h>
203 #include <sys/ioctl.h>
204 #include <sys/uio.h>
205 #include <sys/sysmacros.h>
206 #include <sys/statvfs.h>
207 #include <sys/fsuid.h>
208 #include <sys/xattr.h>
209 #include <sys/sendfile.h>
210 #include <sys/mount.h>
211 #include <sys/quota.h>
212 #include <sys/swap.h>
213 #include <sys/sem.h>
214 #include <sys/shm.h>
215 #include <sys/ipc.h>
216 #include <sys/msg.h>
217 #include <sys/types.h>
218 #include <sys/time.h>
219 #include <sys/resource.h>
220 #include <sys/mman.h>
221 #include <sys/wait.h>
222 #include <sys/socket.h>
223 #include <sys/un.h>
224 #include <sys/poll.h>
225 #include <sys/epoll.h>
226 #include <sys/ioctl.h>
227 #include <sys/uio.h>
228 #include <sys/sysmacros.h>
229 #include <sys/statvfs.h>
230 #include <sys/fsuid.h>
231 #include <sys/xattr.h>
232 #include <sys/sendfile.h>
233 #include <sys/mount.h>
234 #include <sys/quota.h>
235 #include <sys/swap.h>
236 #include <sys/sem.h>
237 #include <sys/shm.h>
238 #include <sys/ipc.h>
239 #include <sys/msg.h>
240 #include <sys/types.h>
241 #include <sys/time.h>
242 #include <sys/resource.h>
243 #include <sys/mman.h>
244 #include <sys/wait.h>
245 #include <sys/socket.h>
246 #include <sys/un.h>
247 #include <sys/poll.h>
248 #include <sys/epoll.h>
249 #include <sys/ioctl.h>
250 #include <sys/uio.h>
251 #include <sys/sysmacros.h>
252 #include <sys/statvfs.h>
253 #include <sys/fsuid.h>
254 #include <sys/xattr.h>
255 #include <sys/sendfile.h>
256 #include <sys/mount.h>
257 #include <sys/quota.h>
258 #include <sys/swap.h>
259 #include <sys/sem.h>
260 #include <sys/shm.h>
261 #include <sys/ipc.h>
262 #include <sys/msg.h>
263 #include <sys/types.h>
264 #include <sys/time.h>
265 #include <sys/resource.h>
266 #include <sys/mman.h>
267 #include <sys/wait.h>
268 #include <sys/socket.h>
269 #include <sys/un.h>
270 #include <sys/poll.h>
271 #include <sys/epoll.h>
272 #include <sys/ioctl.h>
273 #include <sys/uio.h>
274 #include <sys/sysmacros.h>
275 #include <sys/statvfs.h>
276 #include <sys/fsuid.h>
277 #include <sys/xattr.h>
278 #include <sys/sendfile.h>
279 #include <sys/mount.h>
280 #include <sys/quota.h>
281 #include <sys/swap.h>
282 #include <sys/sem.h>
283 #include <sys/shm.h>
284 #include <sys/ipc.h>
285 #include <sys/msg.h>
286 #include <sys/types.h>
287 #include <sys/time.h>
288 #include <sys/resource.h>
289 #include <sys/mman.h>
290 #include <sys/wait.h>
291 #include <sys/socket.h>
29
```

[illegible]

```

58 // tambahkan node 4 di posisi 4
59 System.out.println("tambah node 4 di posisi 4: ");
60 int data2 = 4;
61 int pos = 4;
62 head_3026 = insertAtPosition(head_3026, pos, data2);
63 printList(head_3026);
64 }

```

2.2.4 Hasil Output

```
Console x Install Java 26 Support
[terminated] InsertDLL_2511533026 [Java Application] D:\eclipse\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe
DLL Awal: 2 <-> 3 <-> 5 <->
simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 ditambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 d posisi 4: 1 <-> 2 <-> 4 <-> 3 <-> 5 <-> 6 <->
```

2.2.5 Analisis

Insert di awal sangat efisien ($O(1)$). Insert di posisi tertentu membutuhkan $O(n)$ karena traversal, namun pointer prev mempermudah penyambungan node.

2.3 PenelusuranDLL_2511533026

2.3.1 Teori

DLL mendukung traversal dua arah: forward (menggunakan next) dan backward (menggunakan prev).

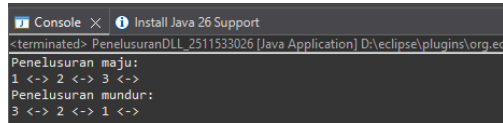
2.3.2 Langkah-langkah

1. Implementasi forwardTraversal.
2. Implementasi backwardTraversal.
3. Testing pada main method.

2.3.3 Contoh Kode

```
1 // Fungsi untuk menambahkan node ke awal
2 void tambahAwal(Node *head, int data) {
3     // Membuat node baru
4     Node *newNode = new Node(data, head, NULL);
5     // Menyambung ke head
6     newNode->next = head;
7     // Mengubah head menjadi node baru
8     head = newNode;
9 }
10 // Fungsi untuk menambahkan node ke akhir
11 void tambahAkhir(Node *head, int data) {
12     // Membuat node baru
13     Node *newNode = new Node(data, NULL, NULL);
14     // Traversal ke akhir
15     Node *temp = head;
16     while (temp->next != NULL) {
17         temp = temp->next;
18     }
19     // Menyambung ke akhir
20     temp->next = newNode;
21 }
22 // Fungsi untuk menambahkan node di posisi tertentu
23 void tambahDiPosisi(Node *head, int data, int posisi) {
24     // Membuat node baru
25     Node *newNode = new Node(data, NULL, NULL);
26     // Traversal ke posisi
27     Node *temp = head;
28     int count = 1;
29     while (count < posisi) {
30         temp = temp->next;
31         count++;
32     }
33     // Menyambung ke posisi
34     newNode->next = temp->next;
35     temp->next = newNode;
36 }
37 // Fungsi untuk menampilkan DLL
38 void tampilkanDLL(Node *head) {
39     if (head == NULL) {
40         cout << "DLL Kosong" << endl;
41         return;
42     }
43     System.out.println("Penelusuran maju:");
44     forwardTraversal(head);
45     System.out.println("Penelusuran mundur: ");
46     backwardTraversal(head);
47 }
48 }
49 }
```

2.3.4 Hasil Output



```
Console x Install Java 26 Support
<terminated> PenelusuranDLL_2511533026 [Java Application] D:\eclipse\plugins\org.ec
Penelusuran maju:
1 <-> 2 <-> 3 <->
Penelusuran mundur:
3 <-> 2 <-> 1 <->
```

2.3.5 Analisis

Kemampuan traversal mundur merupakan keunggulan utama DLL dibandingkan SLL, berguna untuk aplikasi yang memerlukan akses data dari belakang.

2.4 HapusDLL_2511533026

2.4.1 Teori

Operasi delete pada DLL lebih mudah karena setiap node memiliki pointer ke node sebelumnya, sehingga tidak perlu pointer tambahan saat menghapus node tengah.

2.4.2 Langkah-langkah

1. Implementasi delHead.
2. Implementasi delLast.
3. Implementasi delPos.

2.4.3 Contoh Kode

```

1 public class Hw02_211310001 {
2     // 2021년 12월 21일 화요일
3     public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
4     // (test == null) ?
5     // return null;
6     //
7     // Hw02_211310001 head = head;
8     // head = head.next.next;
9     // (test != null) ?
10    // head.next = null;
11    // return head;
12    //
13    // // 2021년 12월 21일 화요일
14    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
15    // (test == null) ?
16    // if (test.next.next == null) ?
17    // return null;
18    //
19    // Hw02_211310001 curr = head;
20    // while (curr.next.next != null) {
21    //     curr = curr.next.next;
22    // }
23    // // 2021년 12월 21일 화요일
24    // if (curr.next.next == null) {
25    //     return head;
26    // }
27    //
28    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
29    // (test == null) ?
30    // if (test != null) {
31    //     return head;
32    // }
33    //
34    // // 2021년 12월 21일 화요일
35    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
36    // (test == null) ?
37    // if (test != null) {
38    //     return head;
39    // }
40    //
41    // // 2021년 12월 21일 화요일
42    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
43    // (test == null) ?
44    // if (test != null) {
45    //     return head;
46    // }
47    //
48    // // 2021년 12월 21일 화요일
49    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
50    // (test == null) ?
51    // if (test != null) {
52    //     return head;
53    // }
54    //
55    // // 2021년 12월 21일 화요일
56    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
57    // (test == null) ?
58    // if (test != null) {
59    //     return head;
60    // }
61    //
62    // // 2021년 12월 21일 화요일
63    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
64    // (test == null) ?
65    // if (test != null) {
66    //     return head;
67    // }
68    //
69    // // 2021년 12월 21일 화요일
70    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
71    // (test == null) ?
72    // if (test != null) {
73    //     return head;
74    // }
75    //
76    // // 2021년 12월 21일 화요일
77    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
78    // (test == null) ?
79    // if (test != null) {
80    //     return head;
81    // }
82    //
83    // // 2021년 12월 21일 화요일
84    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
85    // (test == null) ?
86    // if (test != null) {
87    //     return head;
88    // }
89    //
90    // // 2021년 12월 21일 화요일
91    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
92    // (test == null) ?
93    // if (test != null) {
94    //     return head;
95    // }
96    //
97    // // 2021년 12월 21일 화요일
98    // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
99    // (test == null) ?
100   // if (test != null) {
101   //     return head;
102   // }
103   //
104   // // 2021년 12월 21일 화요일
105   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
106   // (test == null) ?
107   // if (test != null) {
108   //     return head;
109   // }
110   //
111   // // 2021년 12월 21일 화요일
112   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
113   // (test == null) ?
114   // if (test != null) {
115   //     return head;
116   // }
117   //
118   // // 2021년 12월 21일 화요일
119   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
120   // (test == null) ?
121   // if (test != null) {
122   //     return head;
123   // }
124   //
125   // // 2021년 12월 21일 화요일
126   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
127   // (test == null) ?
128   // if (test != null) {
129   //     return head;
130   // }
131   //
132   // // 2021년 12월 21일 화요일
133   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
134   // (test == null) ?
135   // if (test != null) {
136   //     return head;
137   // }
138   //
139   // // 2021년 12월 21일 화요일
140   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
141   // (test == null) ?
142   // if (test != null) {
143   //     return head;
144   // }
145   //
146   // // 2021년 12월 21일 화요일
147   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
148   // (test == null) ?
149   // if (test != null) {
150   //     return head;
151   // }
152   //
153   // // 2021년 12월 21일 화요일
154   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
155   // (test == null) ?
156   // if (test != null) {
157   //     return head;
158   // }
159   //
160   // // 2021년 12월 21일 화요일
161   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
162   // (test == null) ?
163   // if (test != null) {
164   //     return head;
165   // }
166   //
167   // // 2021년 12월 21일 화요일
168   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
169   // (test == null) ?
170   // if (test != null) {
171   //     return head;
172   // }
173   //
174   // // 2021년 12월 21일 화요일
175   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
176   // (test == null) ?
177   // if (test != null) {
178   //     return head;
179   // }
180   //
181   // // 2021년 12월 21일 화요일
182   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
183   // (test == null) ?
184   // if (test != null) {
185   //     return head;
186   // }
187   //
188   // // 2021년 12월 21일 화요일
189   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
190   // (test == null) ?
191   // if (test != null) {
192   //     return head;
193   // }
194   //
195   // // 2021년 12월 21일 화요일
196   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
197   // (test == null) ?
198   // if (test != null) {
199   //     return head;
200   // }
201   //
202   // // 2021년 12월 21일 화요일
203   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
204   // (test == null) ?
205   // if (test != null) {
206   //     return head;
207   // }
208   //
209   // // 2021년 12월 21일 화요일
210   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
211   // (test == null) ?
212   // if (test != null) {
213   //     return head;
214   // }
215   //
216   // // 2021년 12월 21일 화요일
217   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
218   // (test == null) ?
219   // if (test != null) {
220   //     return head;
221   // }
222   //
223   // // 2021년 12월 21일 화요일
224   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
225   // (test == null) ?
226   // if (test != null) {
227   //     return head;
228   // }
229   //
230   // // 2021년 12월 21일 화요일
231   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
232   // (test == null) ?
233   // if (test != null) {
234   //     return head;
235   // }
236   //
237   // // 2021년 12월 21일 화요일
238   // public static Hw02_211310001 hw02_211310001 = new Hw02_211310001();
239   // (test == null) ?
240   // if (test != null) {
24
```

2.4.4 Hasil Output

text

```

> Console
> Install Java 26 Support

<terminated> HapusDLL 2511533026 [Java Application] D:\eclipse\plugins\org.eclipse.jst.j2ee.ui\bin\
DLL Awal: 1 2 3 4 5
Setelah head dihapus: 2 3 4 5
Setelah node terakhir dihapus: menghapus node ke 2: 2 4

```

2.4.5 Analisis

Penghapusan menjadi lebih efisien karena pointer prev memungkinkan update referensi secara langsung tanpa traversal ulang.

2.6 Tugas Pekan 6

2.5.1 Analisis Kode Program

1. Analisis Struktur Data (Double Linked List)

Program ini mengimplementasikan konsep **Double Linked List (DLL)**, sebuah struktur data di mana setiap elemen (*node*) memiliki dua referensi atau *pointer*.

- **Node (Lagu_2511533026):** Berfungsi sebagai kontainer data yang menyimpan judul_3026 dan penyanyi_3026.
- **Pointer next_3026:** Menunjuk ke urutan lagu berikutnya dalam playlist.
- **Pointer prev_3026:** Menunjuk ke urutan lagu sebelumnya, memungkinkan navigasi dua arah yang tidak bisa dilakukan pada *Single Linked List*.

2. Analisis Logika Operasi (Method)

- **Tampil Maju & Mundur (Traversal):**
 - tampilMaju_3026: Menelusuri list mulai dari head menggunakan pointer next hingga mencapai null.
 - tampilMundur_3026: Menelusuri list mulai dari tail menggunakan pointer prev hingga mencapai null. Ini adalah fitur khas DLL yang diwajibkan dalam tugas.
- **Cari Lagu (Search):** Metode cariLagu_3026 melakukan perbandingan *String* secara *case-insensitive* (tidak membedakan huruf besar/kecil) terhadap judul lagu yang dicari.

[illegible]

BAB III

KESIMPULAN

Praktikum pekan ke-6 ini telah berhasil memberikan pemahaman mendalam mengenai Doubly Linked List. Kami telah mempelajari cara membuat node dengan dua pointer, melakukan operasi insert di berbagai posisi, penelusuran maju dan mundur, serta operasi penghapusan secara efisien. Seluruh operasi dasar DLL dapat diimplementasikan dengan baik menggunakan Java.

Dibandingkan dengan Singly Linked List, Doubly Linked List menawarkan fleksibilitas yang jauh lebih tinggi, terutama untuk aplikasi yang memerlukan akses data dari dua arah. Pengetahuan ini akan menjadi dasar yang sangat berguna untuk mempelajari struktur data lanjutan seperti Circular Doubly Linked List dan Deque pada pertemuan berikutnya.

DAFTAR PUSTAKA

1. Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in Java*. Pearson.
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. Wiley.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms*. Addison-Wesley.
4. Materi Kuliah Struktur Data – Universitas Andalas.
5. Dokumentasi Java Oracle.